

1. Time Series Classification

An interesting task in machine learning is classification of time series. In this problem, we will classify the activities of humans based on time series obtained by a Wireless Sensor Network.

- (a) Download the AReM data from: <https://archive.ics.uci.edu/ml/datasets/Activity+Recognition+system+based+on+Multisensor+data+fusion+%28AReM%29>. The dataset contains 7 folders that represent seven types of activities. In each folder, there are multiple files each of which represents an instant of a human performing an activity.¹ Each file contains 6 time series collected from activities of the same person, which are called avg_rss12, var_rss12, avg_rss13, var_rss13, vg_rss23, and ar_rss23. There are 88 instances in the dataset, each of which contains 6 time series and each time series has 480 consecutive values.

- (b) Keep datasets 1 and 2 in folders bending1 and bending 2, as well as datasets 1, 2, and 3 in other folders as test data and other datasets as train data.

- (c) Feature Extraction

Classification of time series usually needs extracting features from them. In this problem, we focus on time-domain features.

- Research what types of time-domain features are usually used in time series classification and list them (examples are minimum, maximum, mean, etc).
- Extract the time-domain features minimum, maximum, mean, median, standard deviation, first quartile, and third quartile for all of the 6 time series in each instance. You are free to normalize/standardize features or use them directly.²

Your new dataset will look like this:

Instance	min ₁	max ₁	mean ₁	median ₁	...	1st quart ₆	3rd quart ₆
1							
2							
3							
⋮	⋮	⋮	⋮	⋮	...	⋮	⋮
88							

where, for example, 1st quart₆, means the first quartile of the sixth time series in each of the 88 instances.

- Estimate the standard deviation of each of the time-domain features you extracted from the data. Then, use Python's bootstrapped or any other method to build a 90% bootstrap confidence interval for the standard deviation of each feature.
- Use your judgement to select the three most important time-domain features (one option may be min, mean, and max).

¹Some of the data files need very minor cleaning. You can do it by Excel or Python.

²You are welcome to experiment to see if they make a difference.

(d) Binary Classification Using Logistic Regression³

- i. Assume that you want to use the training set to classify bending from other activities, i.e. you have a binary classification problem. Depict scatter plots of the features you specified in 1(c)iv extracted from time series 1, 2, and 6 of each instance, and use color to distinguish bending vs. other activities. (See p. 129 of the textbook).⁴
 - ii. Break each time series in your training set into two (approximately) equal length time series. Now instead of 6 time series for each of the training instances, you have 12 time series for each training instance. Repeat the experiment in 1(d)i, i.e depict scatter plots of the features extracted from both parts of the time series 1,2, and 12. Do you see any considerable difference in the results with those of 1(d)i?
 - iii. Break each time series in your training set into $l \in \{1, 2, \dots, 20\}$ time series of approximately equal length and use logistic regression⁵ to solve the binary classification problem, using time-domain features. Remember that breaking each of the time series does not change the number of instances. It only changes the number of features for each instance. Calculate the p-values for your logistic regression parameters in each model corresponding to each value of l and refit a logistic regression model using your pruned set of features.⁶ Alternatively, you can use backward selection using `sklearn.feature_selection` or `glm` in R. Use 5-fold cross-validation to determine the best value of the pair (l, p) , where p is the number of features used in recursive feature elimination. Explain what the right way and the wrong way are to perform cross-validation in this problem.⁷ Obviously, use the right way! Also, you may encounter the problem of class imbalance, which may make some of your folds not having any instances of the rare class. In such a case, you can use *stratified cross validation*. Research what it means and use it if needed.
- In the following, you can see an example of applying Python's Recursive Feature Elimination, which is a backward selection algorithm, to logistic regression.

```
# Recursive Feature Elimination
from sklearn import datasets
from sklearn.feature_selection import RFE
```

³Some logistic regression packages have a built-in $\mathcal{L}_2$ regularization. To remove the effect of $\mathcal{L}_2$ regularization, set $\lambda = 0$ or set the budget $C \rightarrow \infty$ (i.e. a very large value).

⁴You are welcome to repeat this experiment with other features as well as with time series 3, 4, and 5 in each instance.

⁵If you encountered instability of the logistic regression problem because of linearly separable classes, modify the Max-Iter parameter in logistic regression to stop the algorithm immaturely and prevent from its instability.

⁶R calculates the p-values for logistic regression automatically. One way of calculating them in Python is to call R within Python. There are other ways to obtain the p-values as well.

⁷This is an interesting problem in which the number of features changes depending on the value of the parameter l that is selected via cross validation. Another example of such a problem is Principal Component Regression, where the number of principal components is selected via cross validation.

```

from sklearn.linear_model import LogisticRegression
# load the iris datasets
dataset = datasets.load_iris()
# create a base classifier used to evaluate a subset of attributes
model = LogisticRegression()
# create the RFE model and select 3 attributes
rfe = RFE(model, 3)
rfe = rfe.fit(dataset.data, dataset.target)
# summarize the selection of the attributes
print(rfe.support_)
print(rfe.ranking_)

```

- iv. Report the confusion matrix and show the ROC and AUC for your classifier on train data. Report the parameters of your logistic regression β_i 's as well as the p-values associated with them.
 - v. Test the classifier on the test set. Remember to break the time series in your test set into the same number of time series into which you broke your training set. Remember that the classifier has to be tested using the features extracted from the test set. Compare the accuracy on the test set with the cross-validation accuracy you obtained previously.
 - vi. Do your classes seem to be well-separated to cause instability in calculating logistic regression parameters?
 - vii. From the confusion matrices you obtained, do you see imbalanced classes? If yes, build a logistic regression model based on case-control sampling and adjust its parameters. Report the confusion matrix, ROC, and AUC of the model.
- (e) Binary Classification Using $\mathcal{L}_1$ -penalized logistic regression
- i. Repeat 1(d)iii using $\mathcal{L}_1$ -penalized logistic regression,⁸ i.e. instead of using p-values for variable selection, use $\mathcal{L}_1$ regularization. Note that in this problem, you have to cross-validate for both l , the number of time series into which you break each of your instances, and λ , the weight of $\mathcal{L}_1$ penalty in your logistic regression objective function (or C , the budget). Packages usually perform cross-validation for λ automatically.⁹
 - ii. Compare the $\mathcal{L}_1$ -penalized with variable selection using p-values. Which one performs better? Which one is easier to implement?
- (f) Multi-class Classification (The Realistic Case)
- i. Find the best l in the same way as you found it in 1(e)i to build an $\mathcal{L}_1$ -penalized multinomial regression model to classify all activities in your training set.¹⁰ Report your test error. Research how confusion matrices and ROC

⁸For $\mathcal{L}_1$ -penalized logistic regression, you may want to use normalized/standardized features

⁹Using the package Liblinear is strongly recommended.

¹⁰New versions of scikit learn allow using $\mathcal{L}_1$ -penalty for multinomial regression.

curves are defined for multiclass classification and show them for this problem if possible.¹¹

- ii. Repeat 1(f)i using a Naïve Bayes' classifier. Use both Gaussian and Multinomial priors and compare the results.
 - iii. Which method is better for multi-class classification in this problem?
2. ISLR 3.7.4
 3. ISLR, 4.8.3
 4. ISLR 4.8.7
 5. Extra Practice (you do not need to submit the answers): ISLR 3.7.3, 3.7.5, 4.8.4, 4.8.9

¹¹For example, the pROC package in R does the job.